PyTorch DDP

Danyang Zhuo

Duke University

Some materials are from Hao Zhang's slides

Logistics

- Late policy for programming assignments
  - Every assignment has a 48-hour grace period
  - However, you cannot use any standard STINF reasons to further extend the grace period.

- Potential project ideas are posted on the course website.

Technical Specifications[1]		
	GB200 NVL72	**HGX B200**
Blackwell GPUs | Grace CPUs	72 | 36	8 | 0
CPU Cores	2,592 Arm Neoverse V2 Cores	-
Total FP4 Tensor Core	1,440 petaFLOPS	144 petaFLOPS
Total FP8/FP6 Tensor Core	720 petaFLOPS/petaOPS	72 petaFLOPS/petaOPS
Total Fast Memory	Up to 30TB	Up to 1.4TB
Total Memory Bandwidth	Up to 576TB/s	Up to 62TB/s
Total NVLink Bandwidth	130TB/s	14.4TB/s
	Individual Blackwell GPU Specifications	
FP4 Tensor Core	20 petaFLOPS	18 petaFLOPS
FP8/FP6 Tensor Core	10 petaFLOPS	9 petaFLOPS
INT8 Tensor Core	10 petaOPS	9 petaOPS
FP16/BF16 Tensor Core	5 petaFLOPS	4.5 petaFLOPS
TF32 Tensor Core	2.5 petaFLOPS	2.2 petaFLOPS
FP32	80 teraFLOPS	75 teraFLOPS
FP64/FP64 Tensor Core	40 teraFLOPS	37 teraFLOPS
GPU Memory | Bandwidth	186GB HBM3e | 8 TB/s	180GB HBM3e | 7.7 TB/s
Multi-Instance GPU (MIG)	7	
Decompression Engine	Yes	
Decoders	7 NVDEC[2] 7 NVJPG	
Max Thermal Design Power (TDP)	Configurable up to 1,200W	Configurable up to 1,000W
Interconnect	5th Generation NVLink: 1.8TB/s PCIe Gen5: 128GB/s	
Server Options	NVIDIA GB200 NVL72 partner and NVIDIA-Certified Systems™ with 72 GPUs	NVIDIA HGX B200 partner and NVIDIA-Certified Systems with 8 GPUs

NVIDIA B200 (Blackwell)

Technical Specifications		
	H200 SXM[1]	**H200 NVL**[1]
FP64	34 TFLOPS	30 TFLOPS
FP64 Tensor Core	67 TFLOPS	60 TFLOPS
FP32	67 TFLOPS	60 TFLOPS
TF32 Tensor Core[2]	989 TFLOPS	835 TFLOPS
BFLOAT16 Tensor Core[2]	1,979 TFLOPS	1,671 TFLOPS
FP16 Tensor Core[2]	1,979 TFLOPS	1,671 TFLOPS
FP8 Tensor Core[2]	3,958 TFLOPS	3,341 TFLOPS
INT8 Tensor Core[2]	3,958 TFLOPS	3,341 TFLOPS
GPU Memory	141GB	141GB
GPU Memory Bandwidth	4.8TB/s	4.8TB/s
Decoders	7 NVDEC 7 JPEG	7 NVDEC 7 JPEG
Confidential Computing	Supported	Supported
Max Thermal Design Power (TDP)	Up to 700W (configurable)	Up to 600W (configurable)
Multi-Instance GPUs	Up to 7 MIGs @18GB each	Up to 7 MIGs @16.5GB each
Form Factor	SXM	PCIe Dual-slot air-cooled
Interconnect	NVIDIA NVLink: 900GB/s PCIe Gen5: 128GB/s	2- or 4-way NVIDIA NVLink bridge: 900GB/s per GPU PCIe Gen5: 128GB/s
Server Options	NVIDIA HGX™ H200 partner and NVIDIA-Certified Systems™ with 4 or 8 GPUs	NVIDIA MGX™ H200 NVL partner and NVIDIA-Certified Systems with up to 8 GPUs
NVIDIA AI Enterprise	Add-on	Included

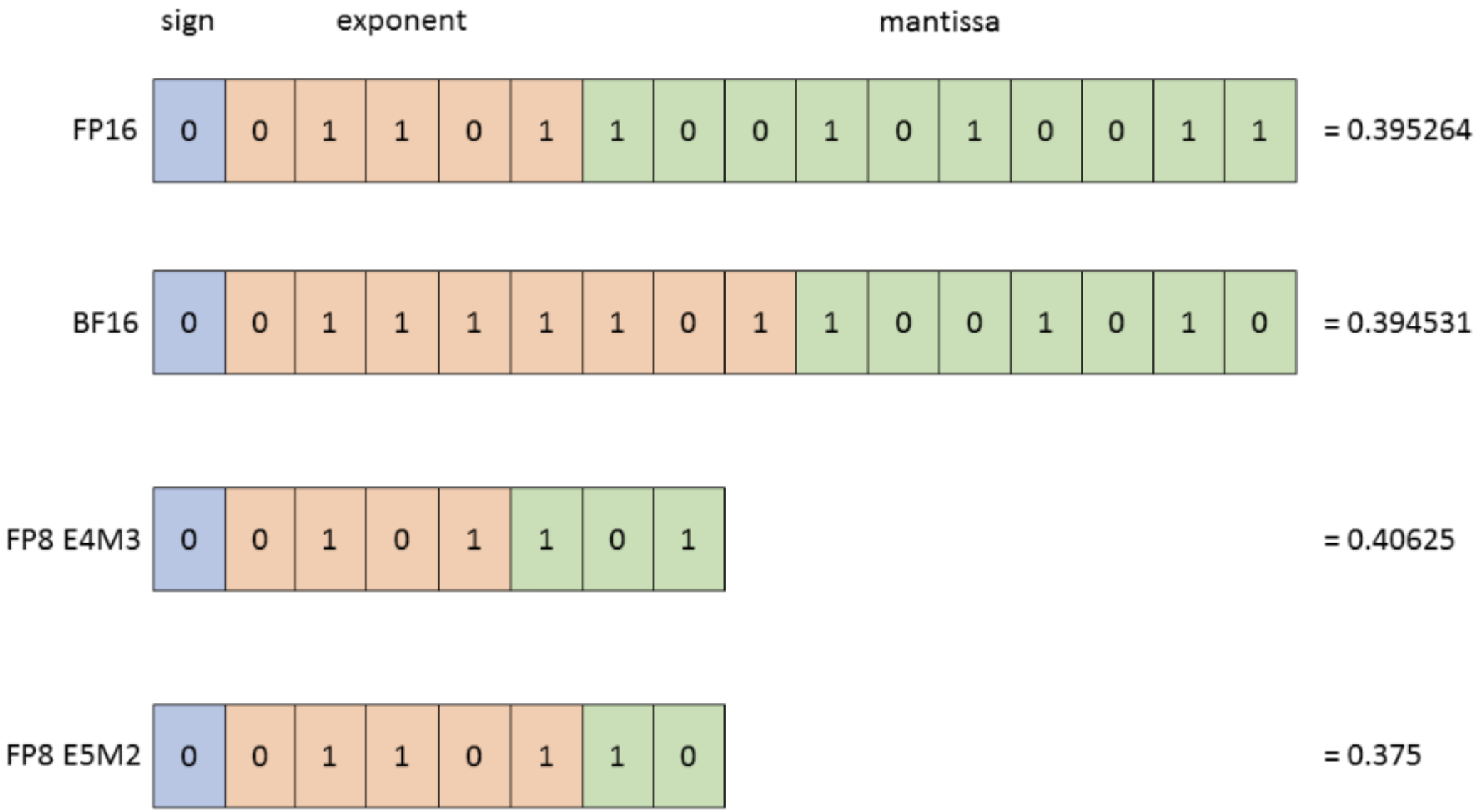
1. Preliminary specifications. May be subject to change.
2. With sparsity.

NVIDIA H200 (Hopper)

NVIDIA A100 TENSOR CORE GPU SPECIFICATIONS (SXM4 AND PCIE FORM FACTORS)

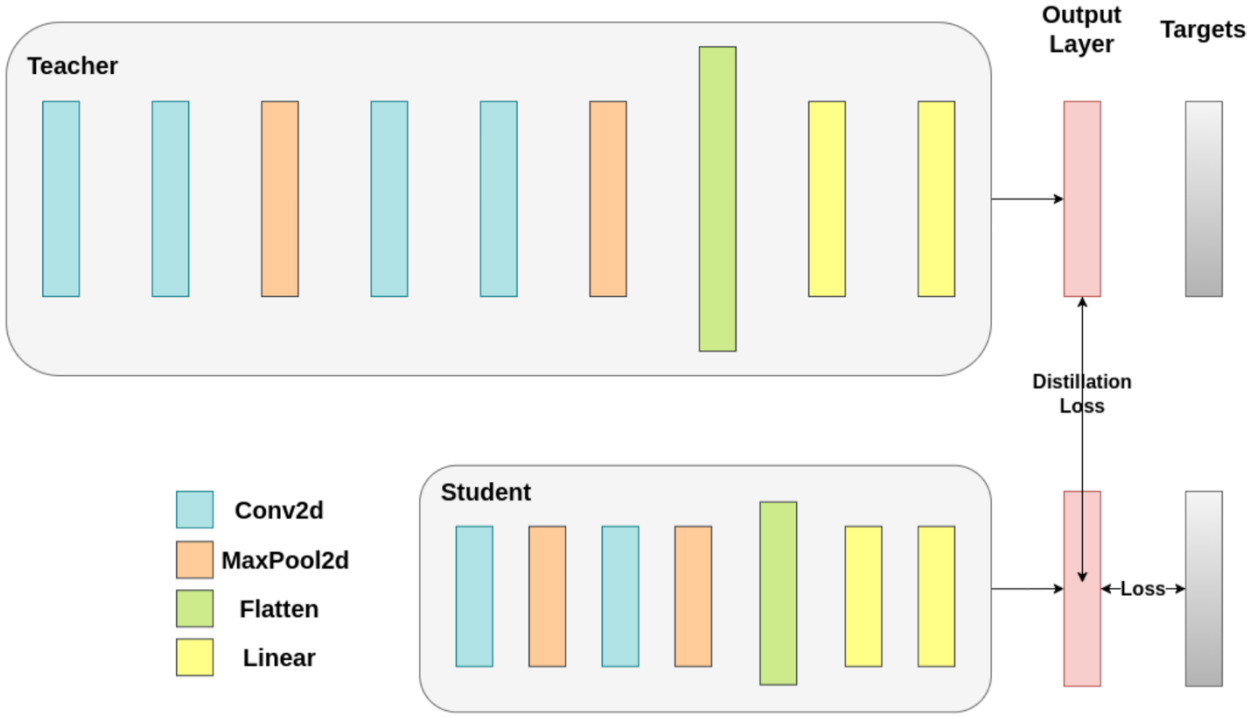
	A100 80GB PCIe	A100 80GB SXM
FP64	9.7 TFLOPS	
FP64 Tensor Core	19.5 TFLOPS	
FP32	19.5 TFLOPS	
Tensor Float 32 (TF32)	156 TFLOPS | 312 TFLOPS*	
BFLOAT16 Tensor Core	312 TFLOPS | 624 TFLOPS*	
FP16 Tensor Core	312 TFLOPS | 624 TFLOPS*	
INT8 Tensor Core	624 TOPS | 1248 TOPS*	
GPU Memory	80GB HBM2e	80GB HBM2e
GPU Memory Bandwidth	1,935GB/s	2,039GB/s

NVIDIA A100 (Ampere)


sign
exponent
mantissa
FP16
0 0 1 1 0 1 1 0 0 1 0 1 0 0 1 1
= 0.395264
BF16
0 0 1 1 1 1 1 0 1 1 0 0 1 0 1 0
= 0.394531
FP8 E4M3
0 0 1 0 1 1 0 1
= 0.40625
FP8 E5M2
0 0 1 1 0 1 1 0
= 0.375

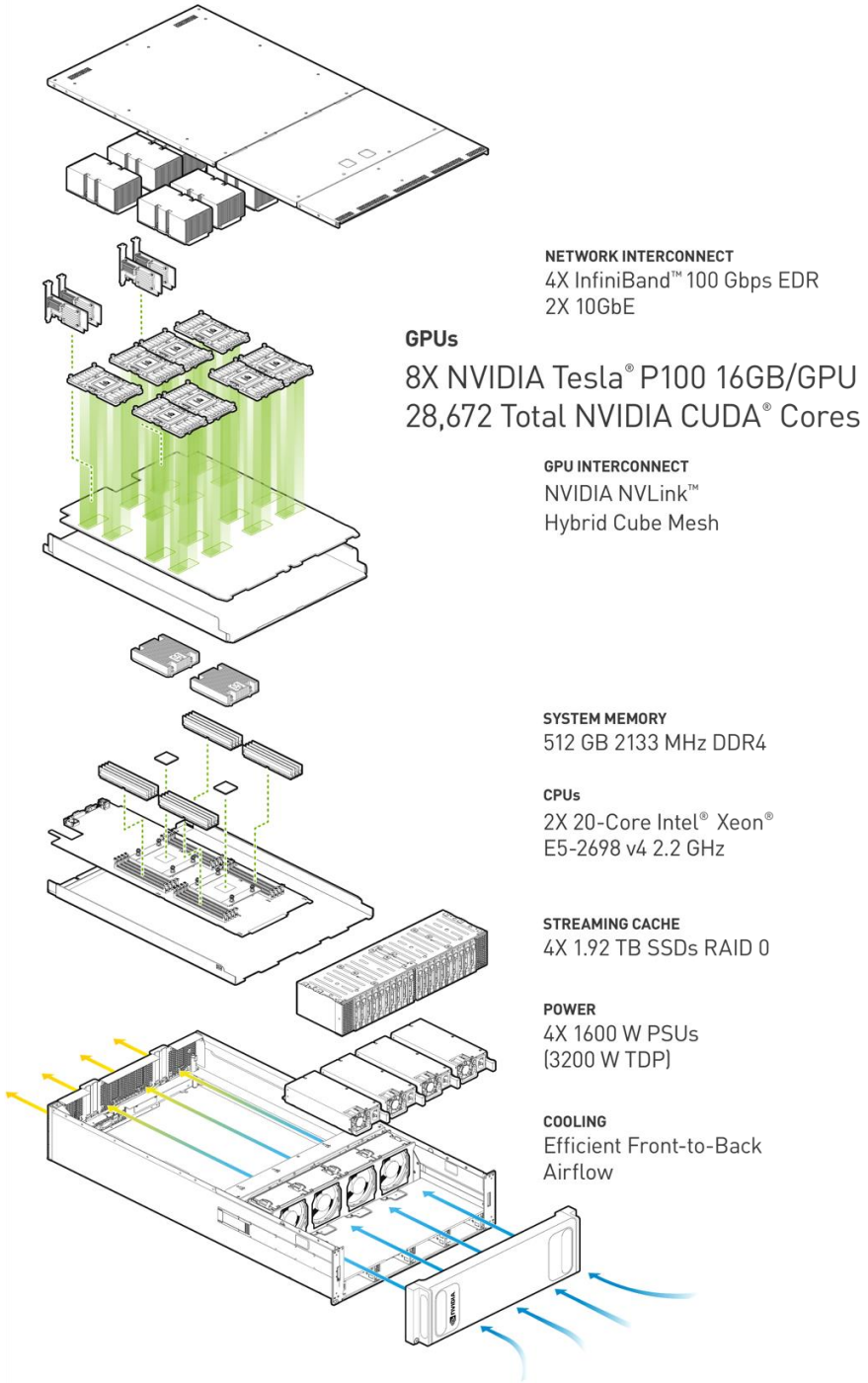
2.4. Distillation: Empower Small Models with Reasoning Capability

To equip more efficient smaller models with reasoning capabilities like DeepSeek-R1, we directly fine-tuned open-source models like Qwen (Qwen, 2024b) and Llama (AI@Meta, 2024) using the 800k samples curated with DeepSeek-R1, as detailed in §2.3.3. Our findings indicate that this straightforward distillation method significantly enhances the reasoning abilities of smaller models. The base models we use here are Qwen2.5-Math-1.5B, Qwen2.5-Math-7B, Qwen2.5-14B, Qwen2.5-32B, Llama-3.1-8B, and Llama-3.3-70B-Instruct. We select Llama-3.3 because its reasoning capability is slightly better than that of Llama-3.1.

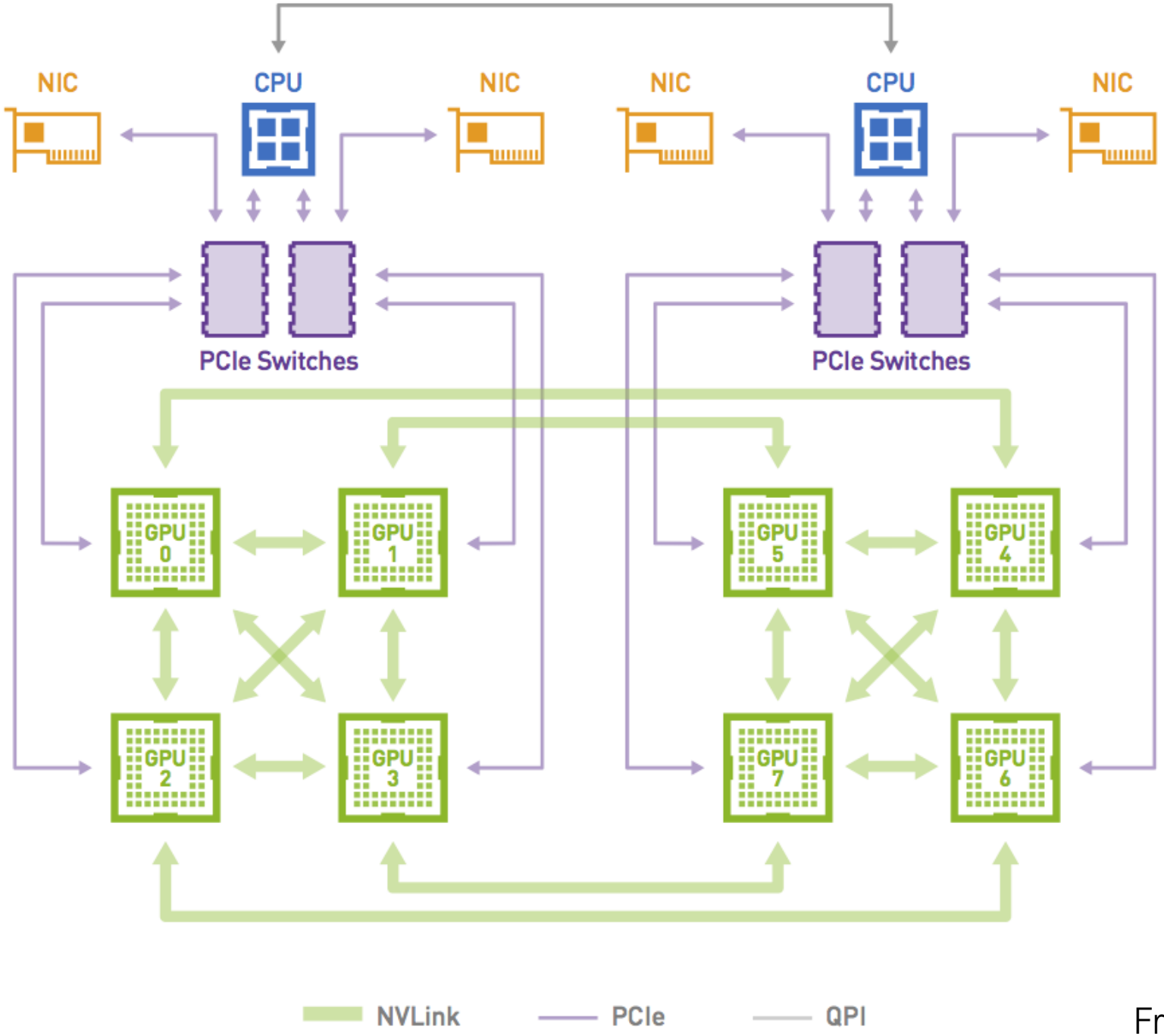


From “DeepSeek-R1”

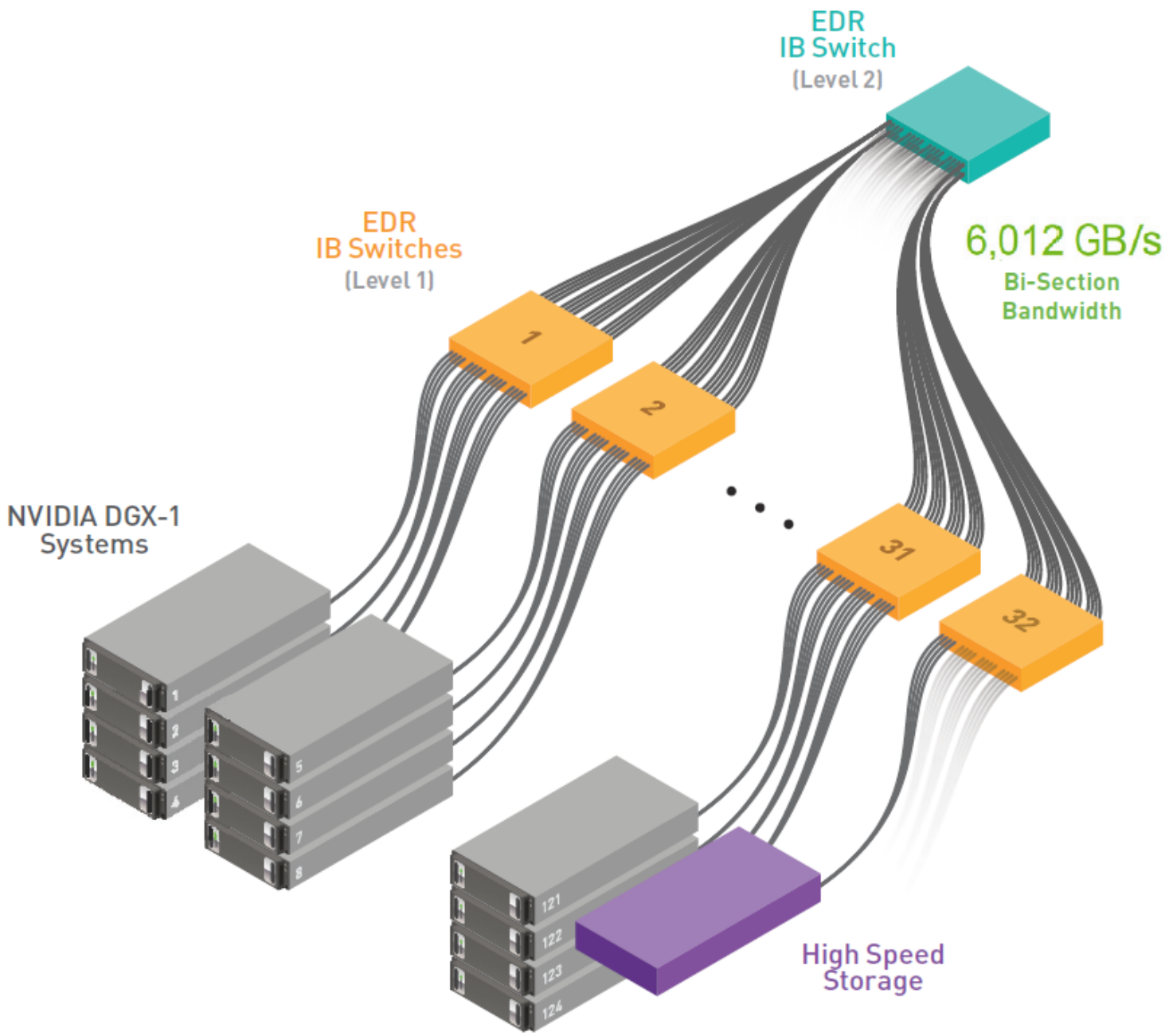
From “PyTorch’s knowledge distillation tutorial”



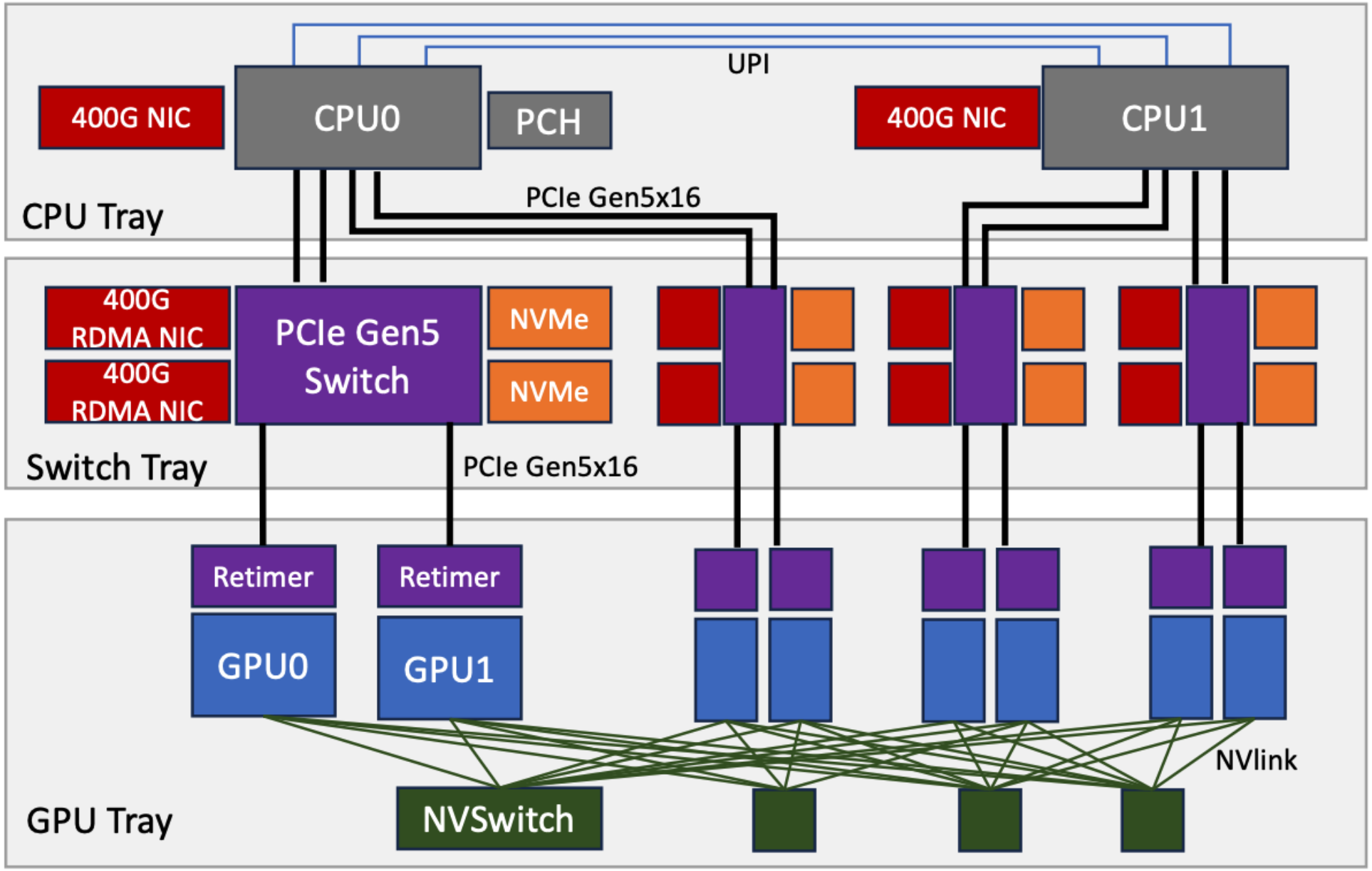
From “NVIDIA DGX-1”



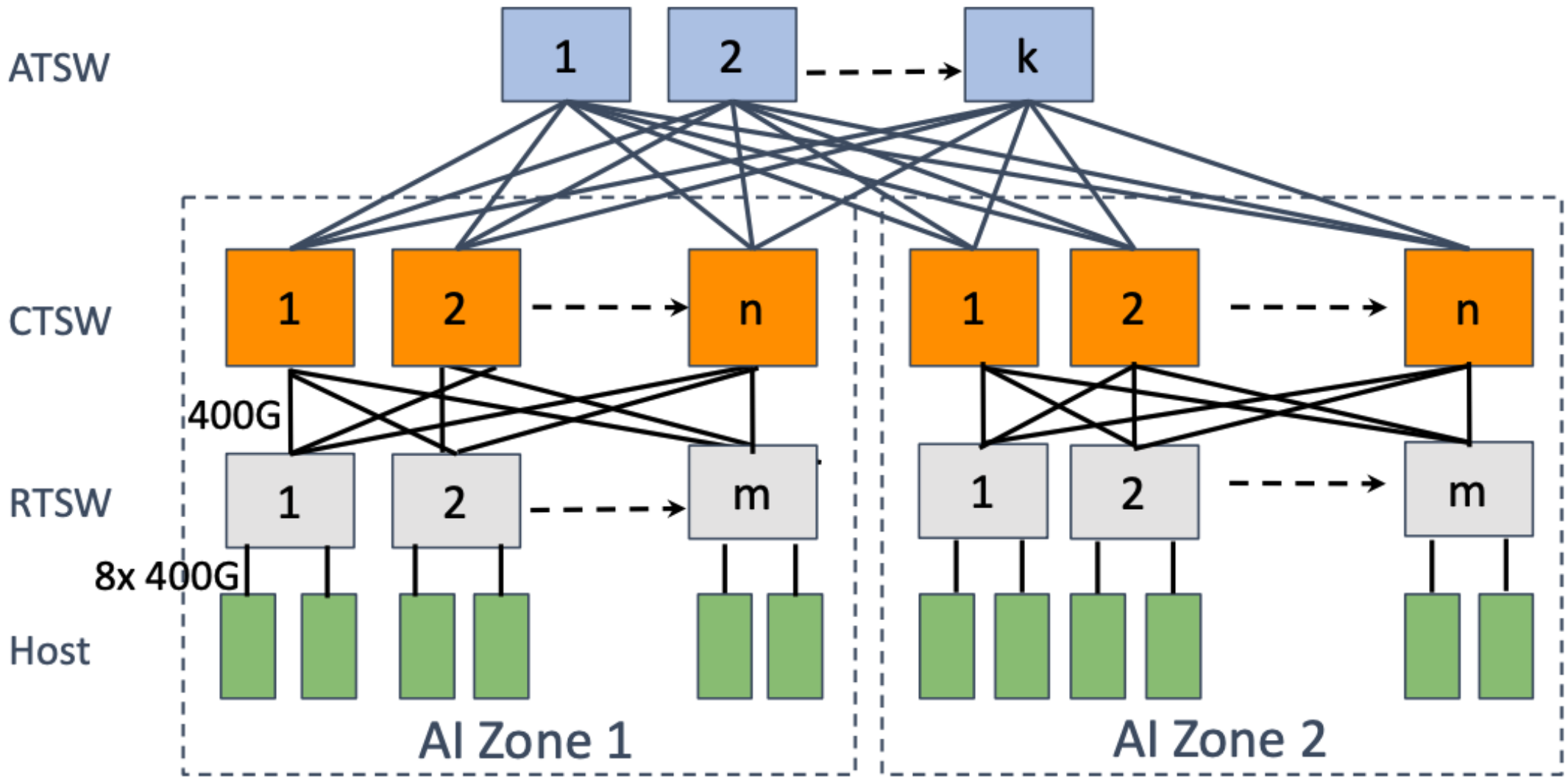
From “NVIDIA DGX-1”



From “NVIDIA DGX-1”



From “RDMA over Ethernet for Distributed AI Training at Meta Scale”

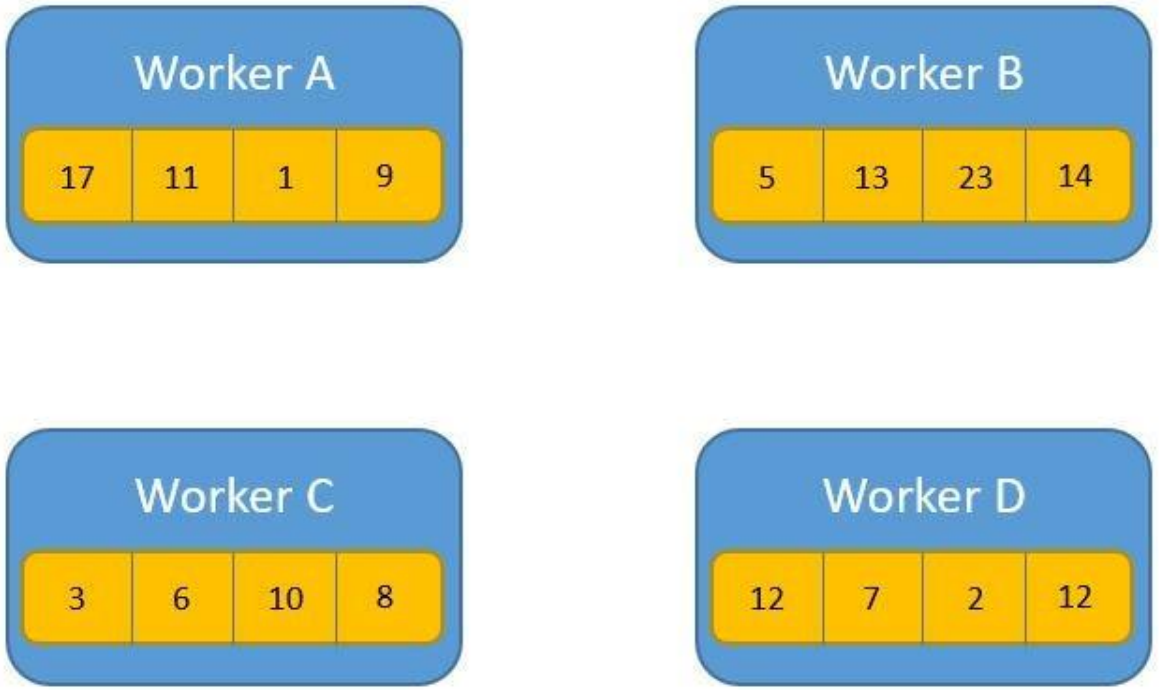


From “RDMA over Ethernet for Distributed AI Training at Meta Scale”

Let’s consider the following case

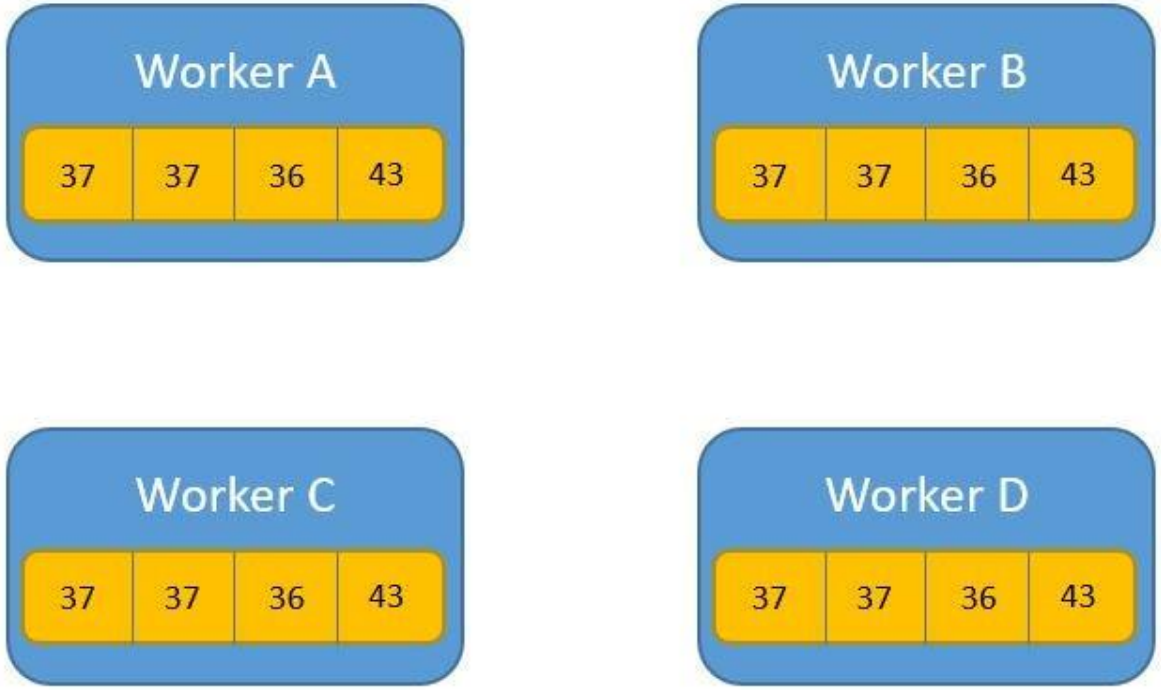
- You partition training data onto four machines, each with a GPU
- In a training iteration, the four GPUs calculated different gradients for the model update.
  - How to synchronize gradients?

AllReduce

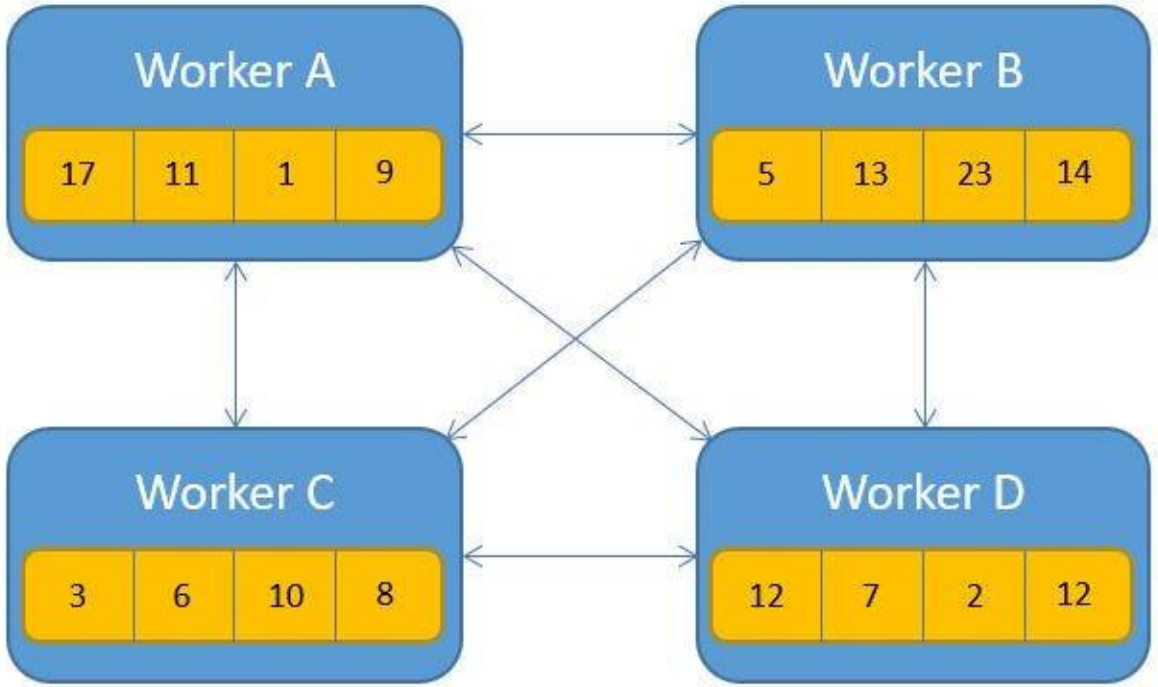


From “Visual intuition on ring-Allreduce for distributed Deep Learning”

AllReduce

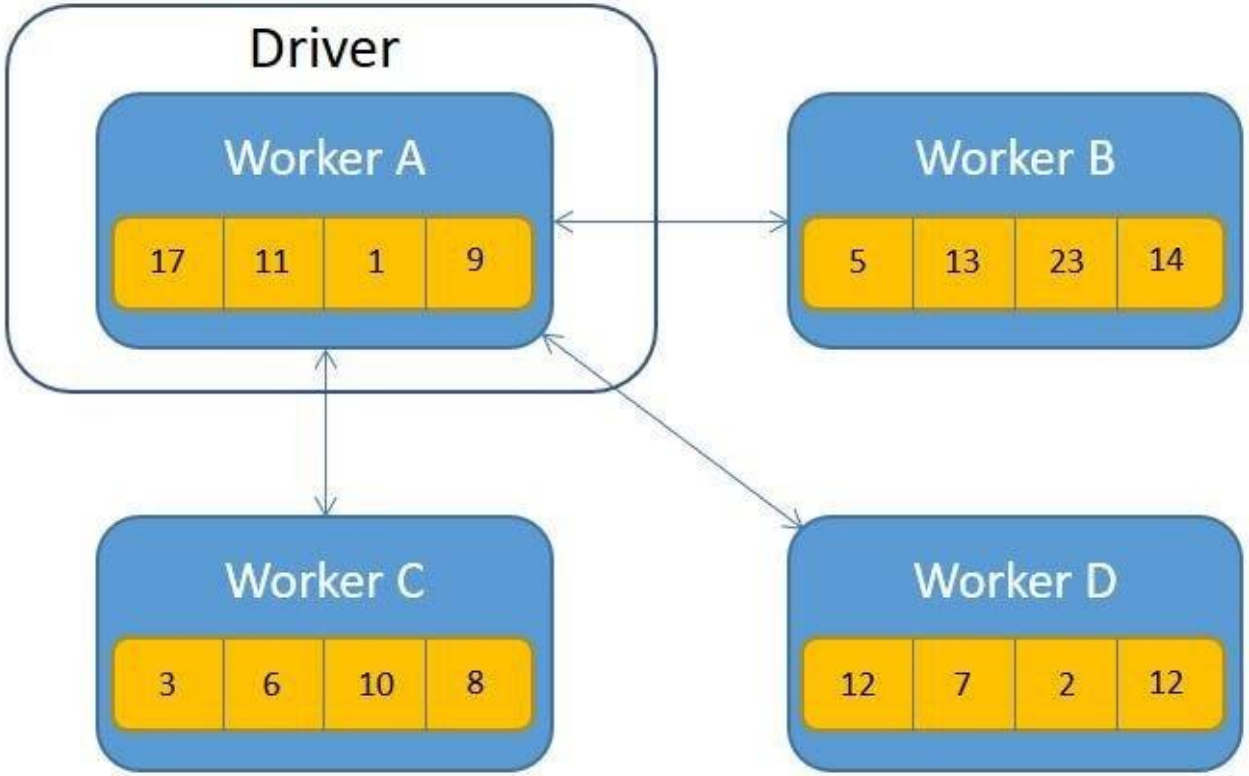

Worker A
37
37
36
43
Worker B
37
37
36
43
Worker C
37
37
36
43
Worker D
37
37
36
43

Naïve Approach #1



Every node sends/receives 3 * 4 = 12 units of data

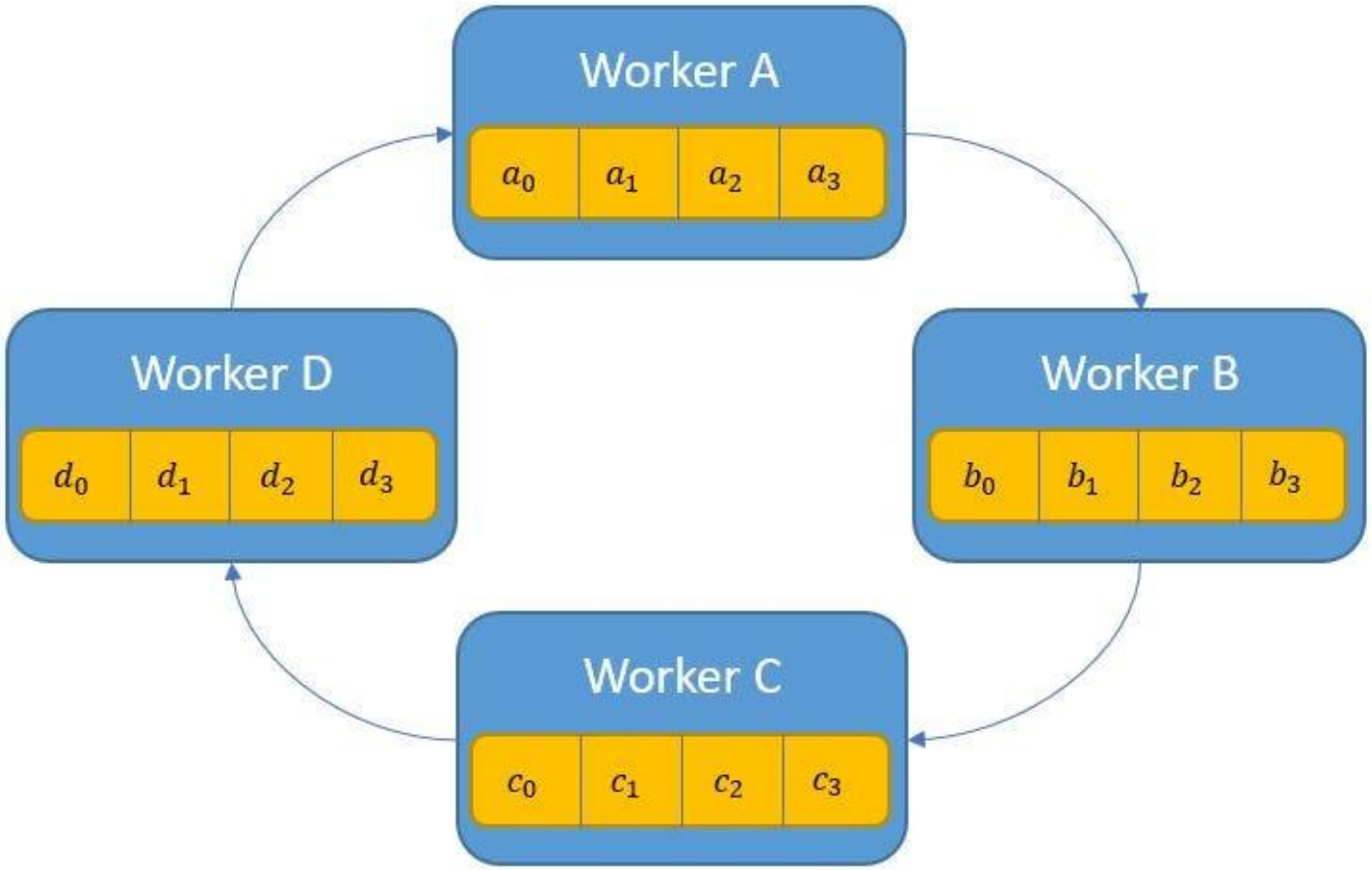
Naïve Approach #2



Driver sends/receives 3 * 4 = 12 units of data
Other nodes send/receive 4 units of data

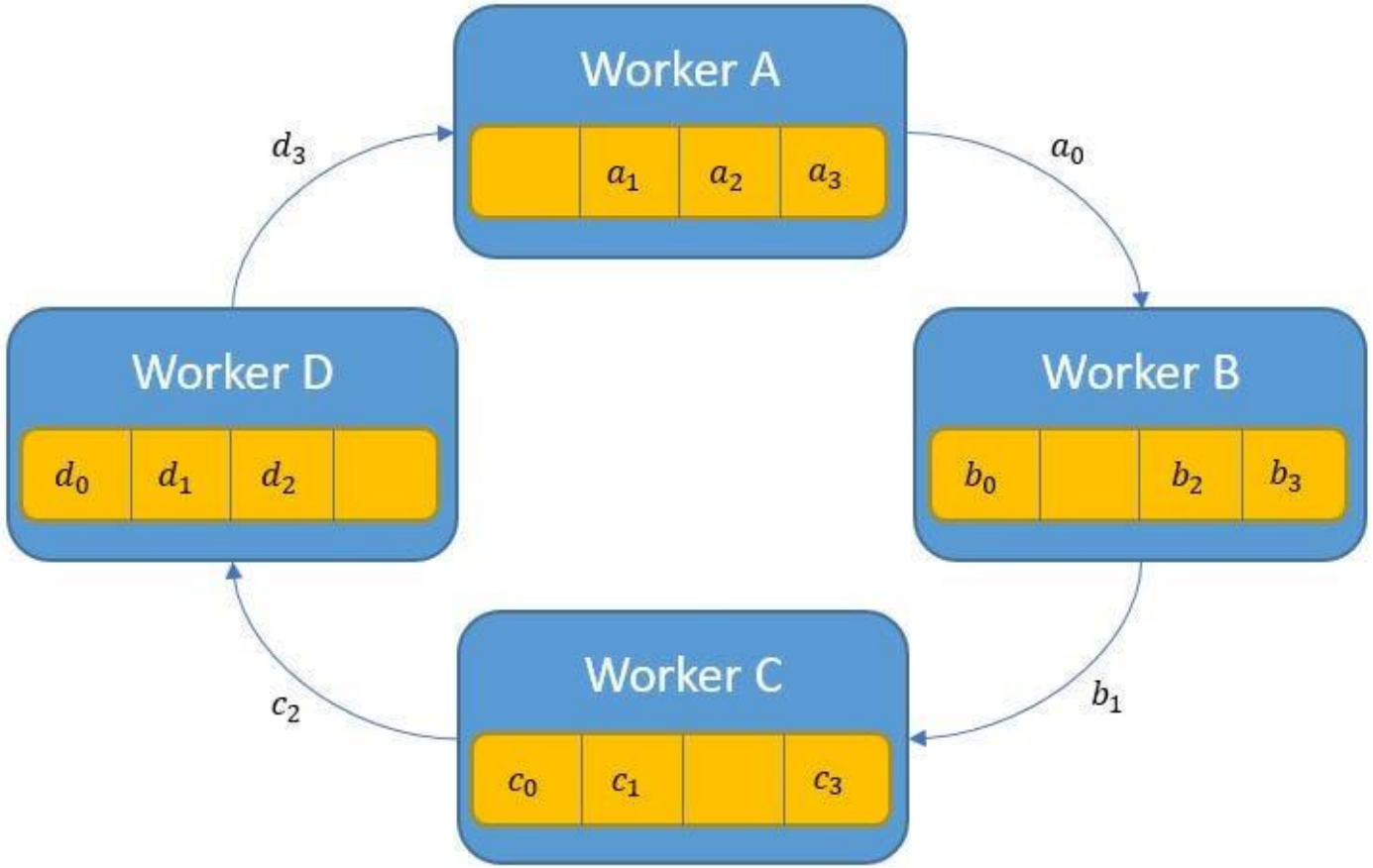
Ring AllReduce

Sends/receives 1 unit of data
Knows (a0, a1, a2, a3)



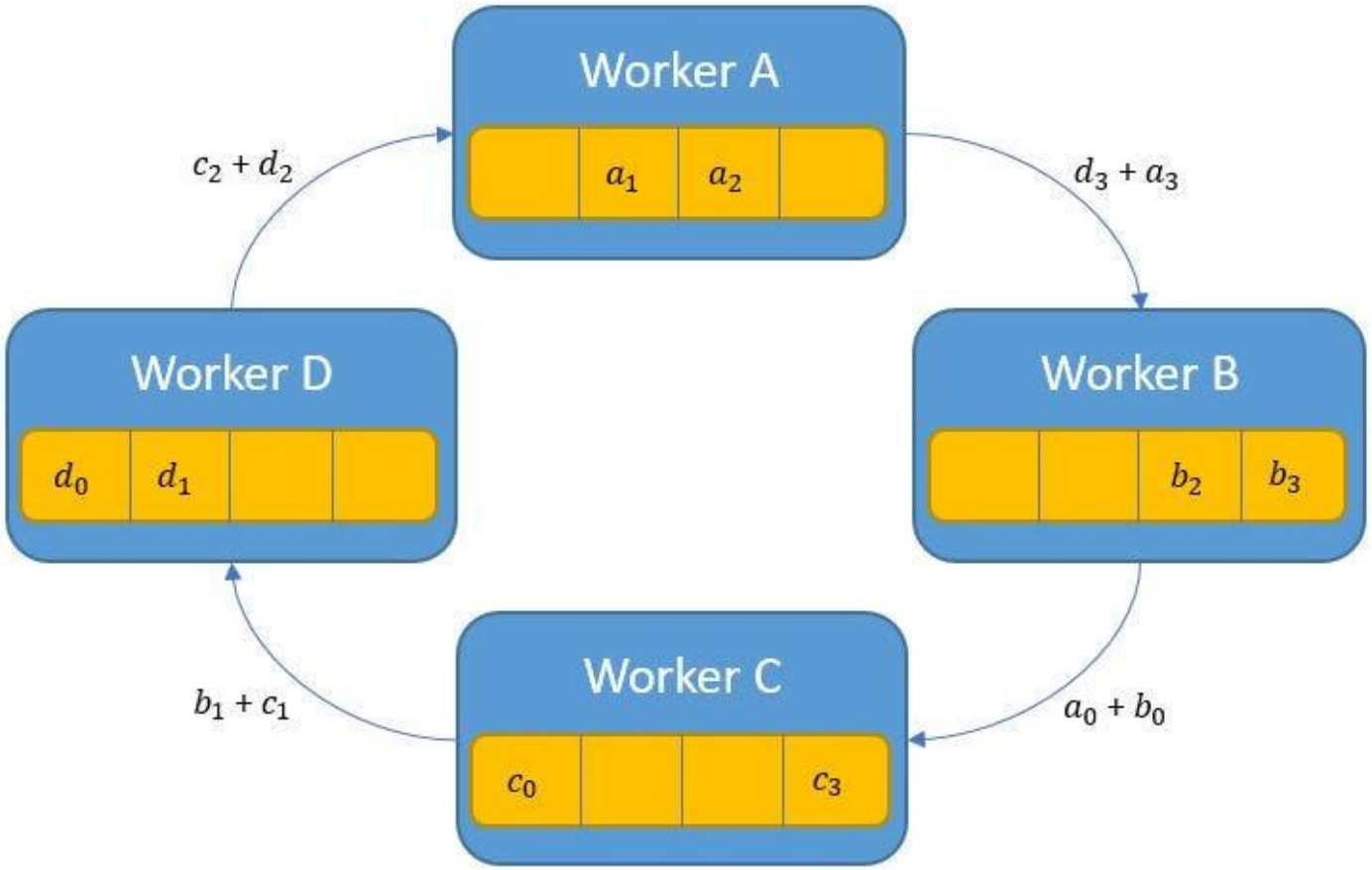
Ring AllReduce

Sends/receives 1 unit of data
Knows (a0, a1, a2, a3 + d3)



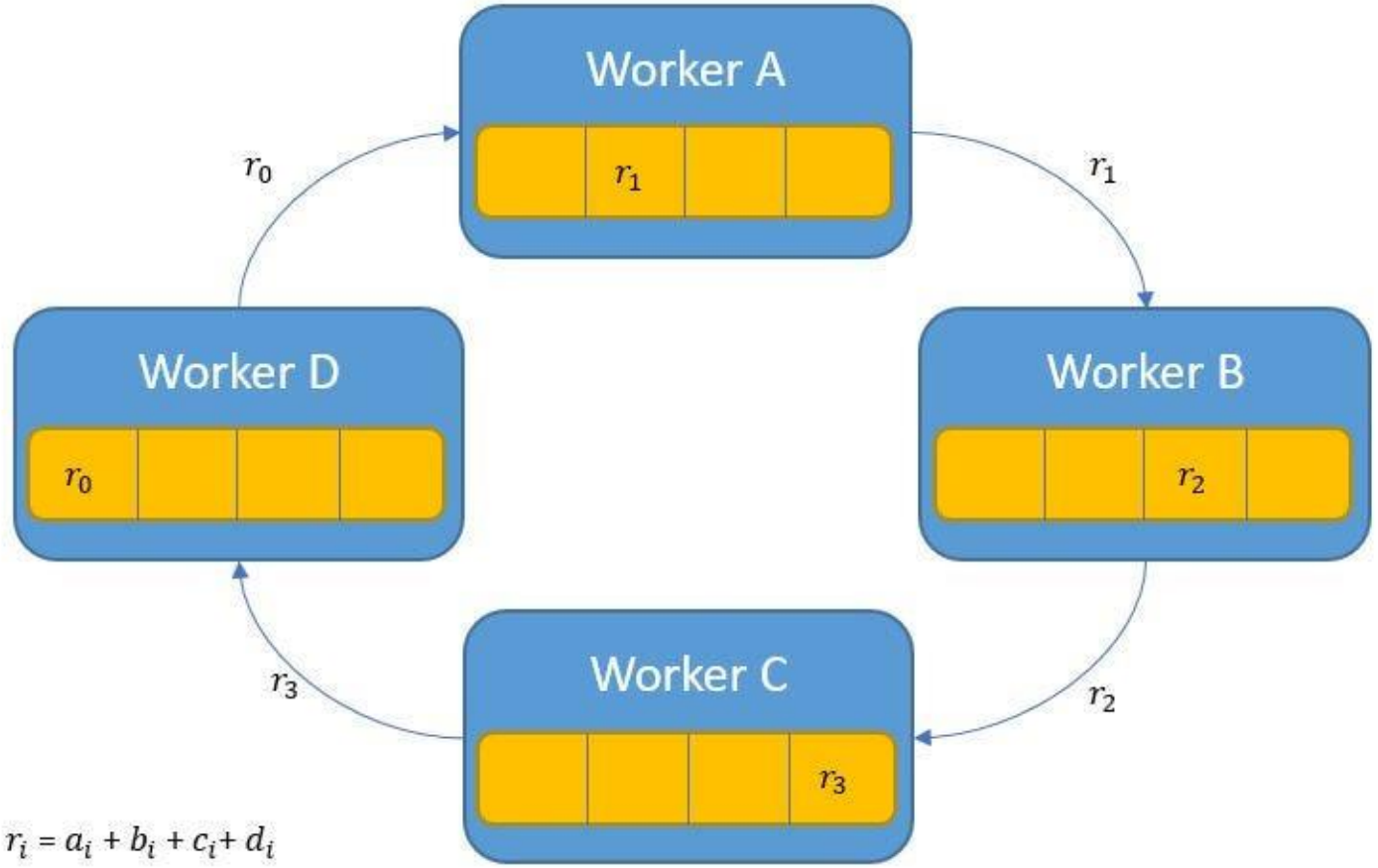
Ring AllReduce

Sends/receives 2 unit of data
Knows (a0, a1, a2 + c2 + d2, a3 + d3)



Ring AllReduce

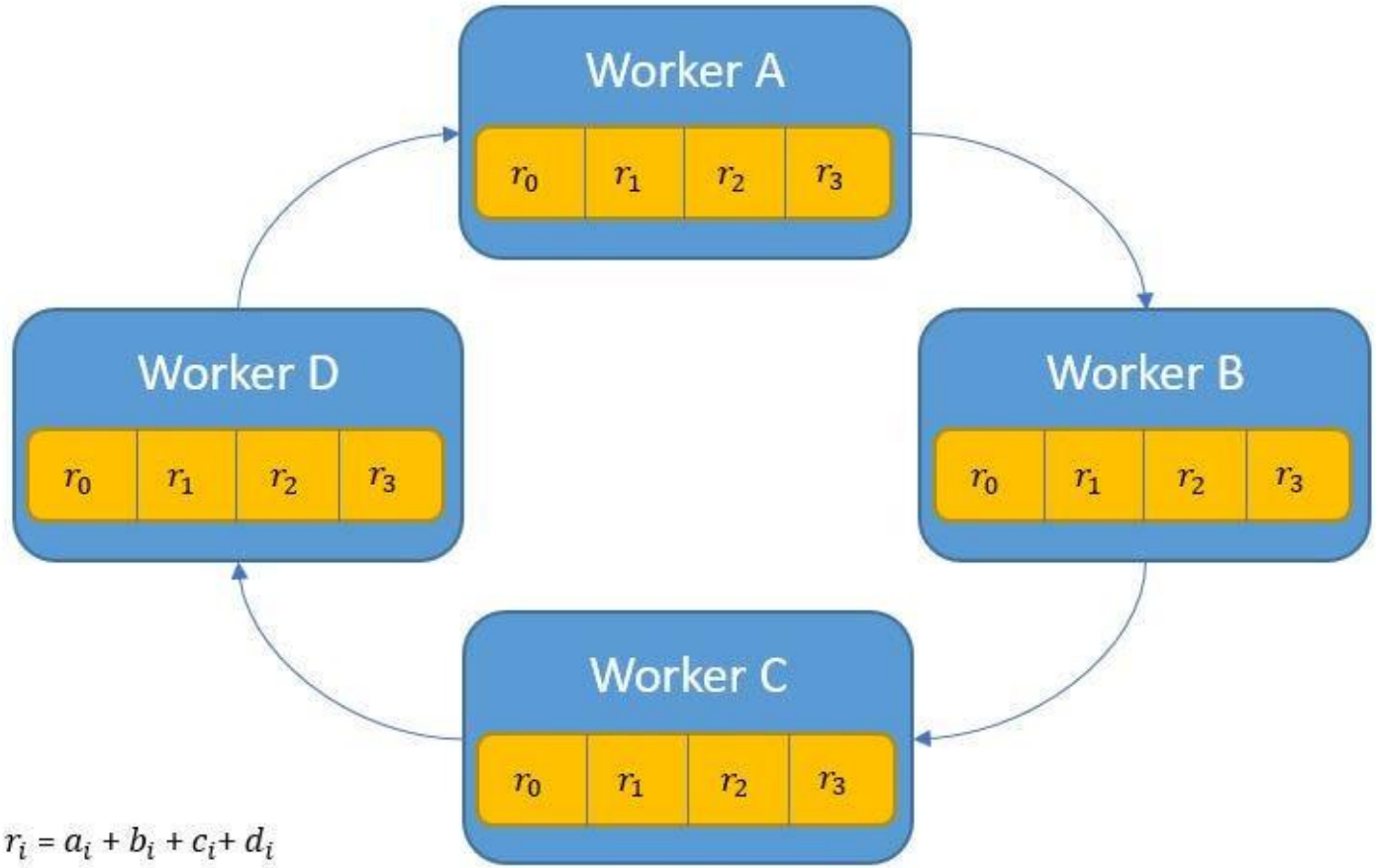
Sends/receives 3 unit of data
Knows (a0, **r1**, a2 + c2 + d2, a3 + d3)



$r_i = a_i + b_i + c_i + d_i$

Ring AllReduce

Sends/receives 6 unit of data
Knows (r0, r1, r2, r3)



$r_i = a_i + b_i + c_i + d_i$

Data parallel

- If you have X amount of data, let’s split the training on N GPUs, each GPU trains on X/N data

Design goals

- Mathematical equivalence
- Non-intrusive and interceptive API
- High performance

AllReduce

- AllReduce is the primitive communication API used by DistributedDataParallel to compute gradient summation across all processes.

- The AllReduce operation expects each participating process to provide an equally-sized tensor, collectively applies a given arithmetic operation (e.g., sum, prod, min, max) to input tensors from all processes, and returns the same result tensor to each participant.

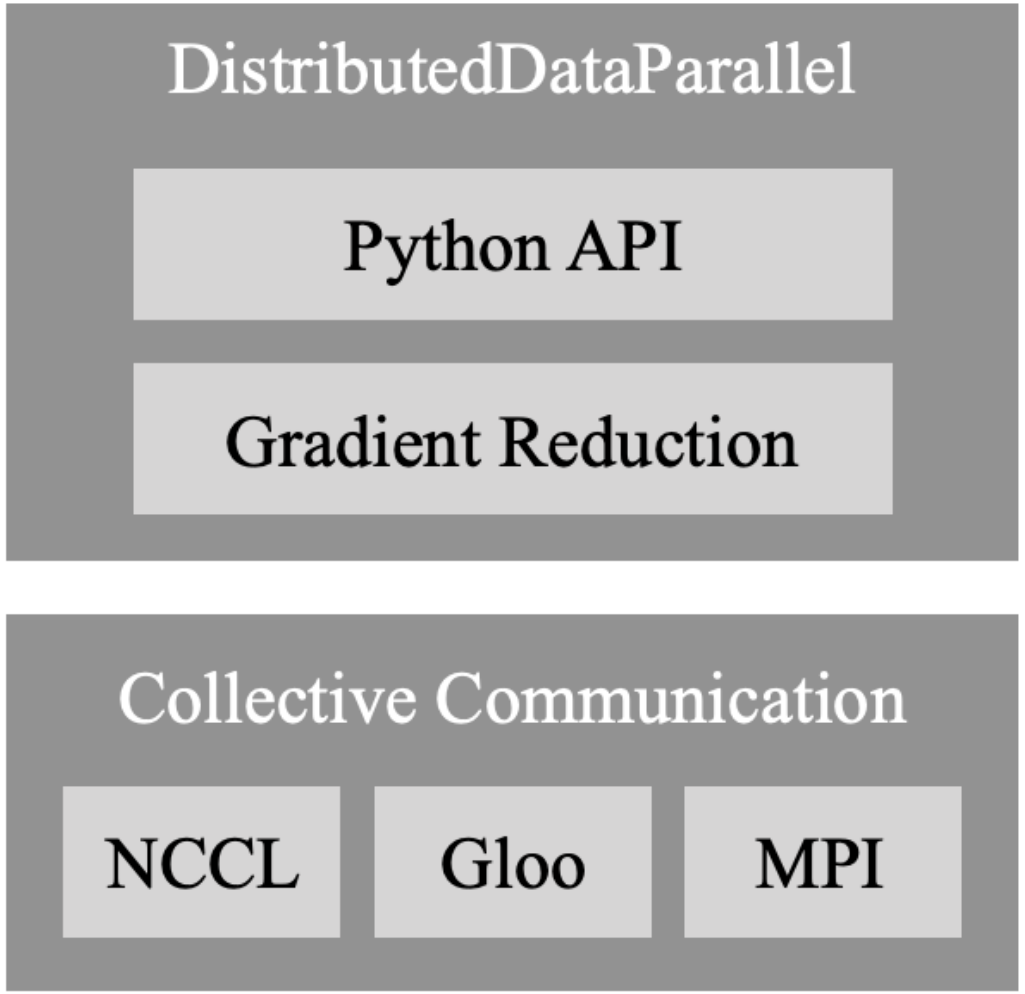


Figure 1: Building Blocks

Mathematical equivalence

- DDP guarantees the correctness by making sure that all model replicas start from the exact same model state, and see the same parameter gradients after every backward pass.

- Therefore, even though optimizers from different processes are all independent, they should be able to bring their local model replicas to the same state at the end of every iteration.

```python
import torch
import torch.nn as nn
import torch.nn.parallel as par
import torch.optim as optim

# initialize torch.distributed properly
# with init_process_group

# setup model and optimizer
net = nn.Linear(10, 10)
net = par.DistributedDataParallel(net)
opt = optim.SGD(net.parameters(), lr=0.01)

# run forward pass
inp = torch.randn(20, 10)
exp = torch.randn(20, 10)
out = net(inp)

# run backward pass
nn.MSELoss()(out, exp).backward()

# update parameters
opt.step()
```

Gradient reduction

- PyTorch autograd engine accepts custom backward hooks.
- DDP can register autograd hooks to trigger computation after every backward pass. When fired, each hook scans through all local model parameters, and retrieves the gradient tensor from each parameter.
- Then, it uses the AllReduce collective communication call to calculate the average gradients on each parameter across all processes, and writes the result back to the gradient tensor.

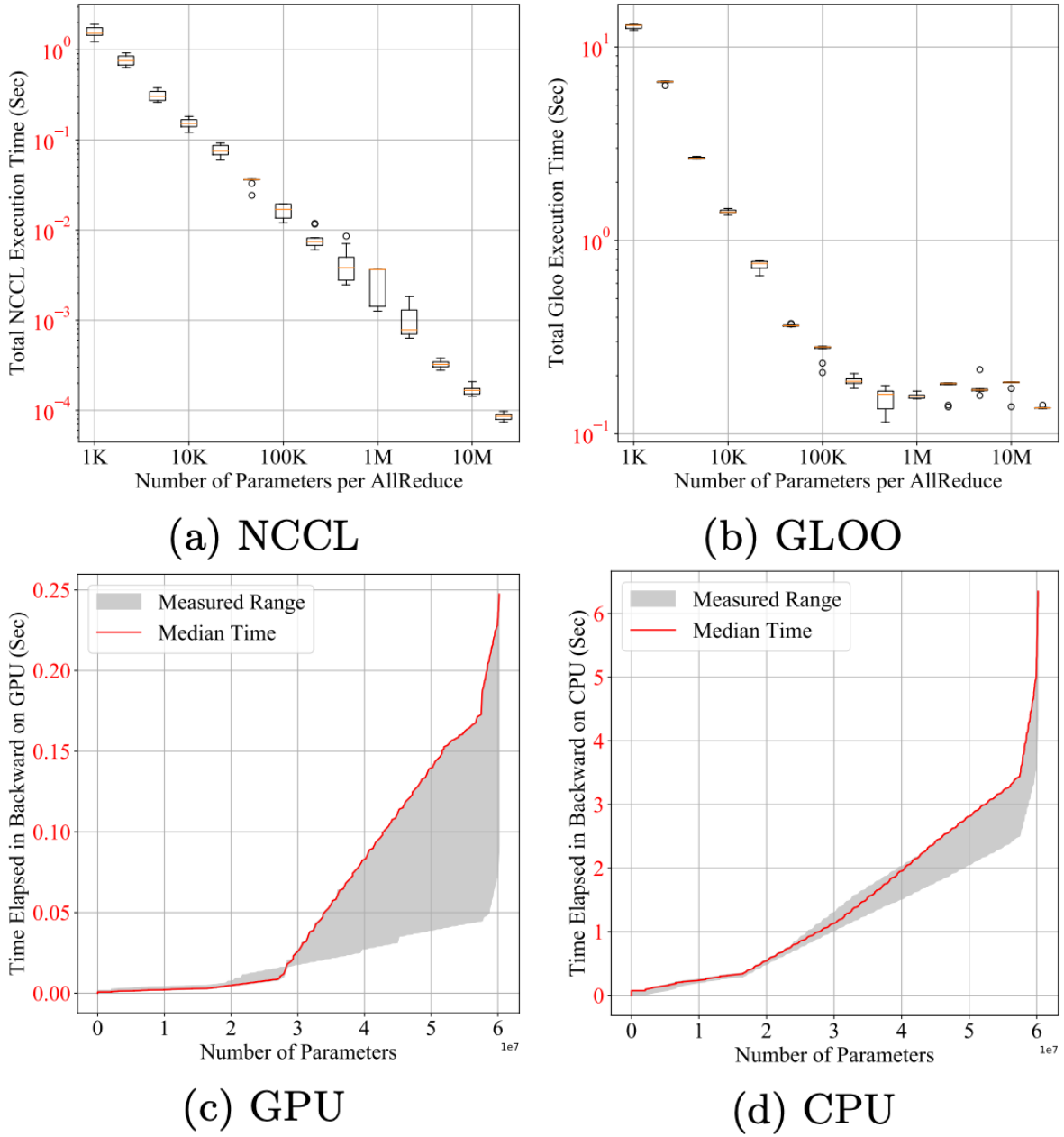


Figure 2: Communication vs Computation Delay

Key techniques in PyTorch DDP

- Gradient bucketing
- Overlap computation and communication
  - Back propagation: B1 -> B2 -> B3 -> B4
  - B2 happens at the same time as AllReduce(Result of B1)
- Techniques to ensure safety

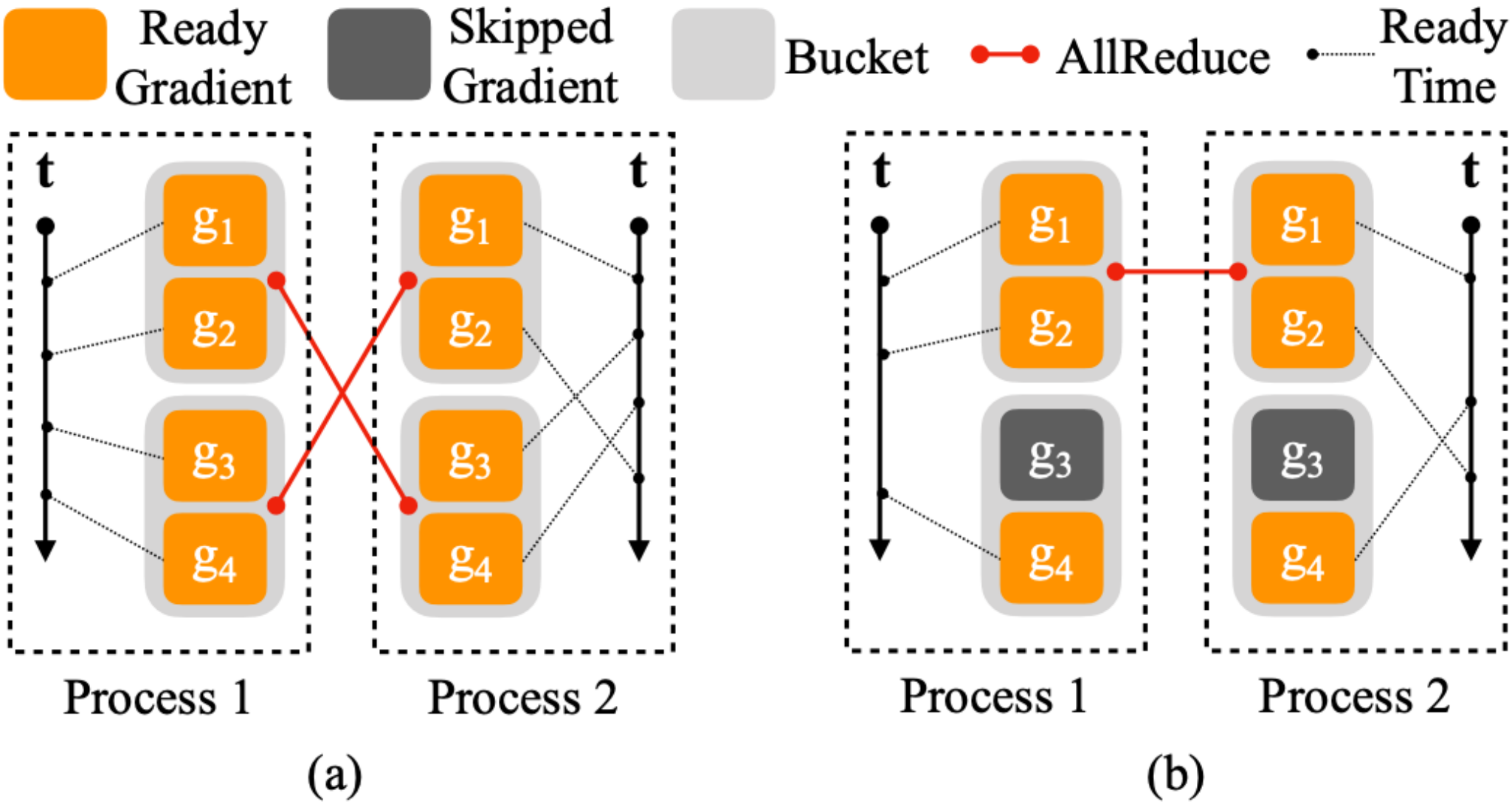


Figure 3: Gradient Synchronization Failures

Algorithm 1: DistributedDataParallel

```
Input: Process rank r, bucket size cap c, local model net
Function constructor(net):
    if r=0 then
        broadcast net states to other processes
    init buckets, allocate parameters to buckets in the reverse order of net.parameters()
    for p in net.parameters() do
        acc ← p.grad_accumulator
        acc → add_post_hook(autograd_hook)
Function forward(inp):
    out = net(inp)
    traverse autograd graph from out and mark unused parameters as ready
    return out
Function autograd_hook(param_index):
    get bucket b_i and bucket offset using param_index
    get parameter var using param_index
    view ← b_i.narrow(offset, var.size())
    view.copy_(var.grad)
    if all grads in b_i are ready then
        mark b_i as ready
    launch AllReduce on ready buckets in order
    if all buckets are ready then
        block waiting for all AllReduce ops
```

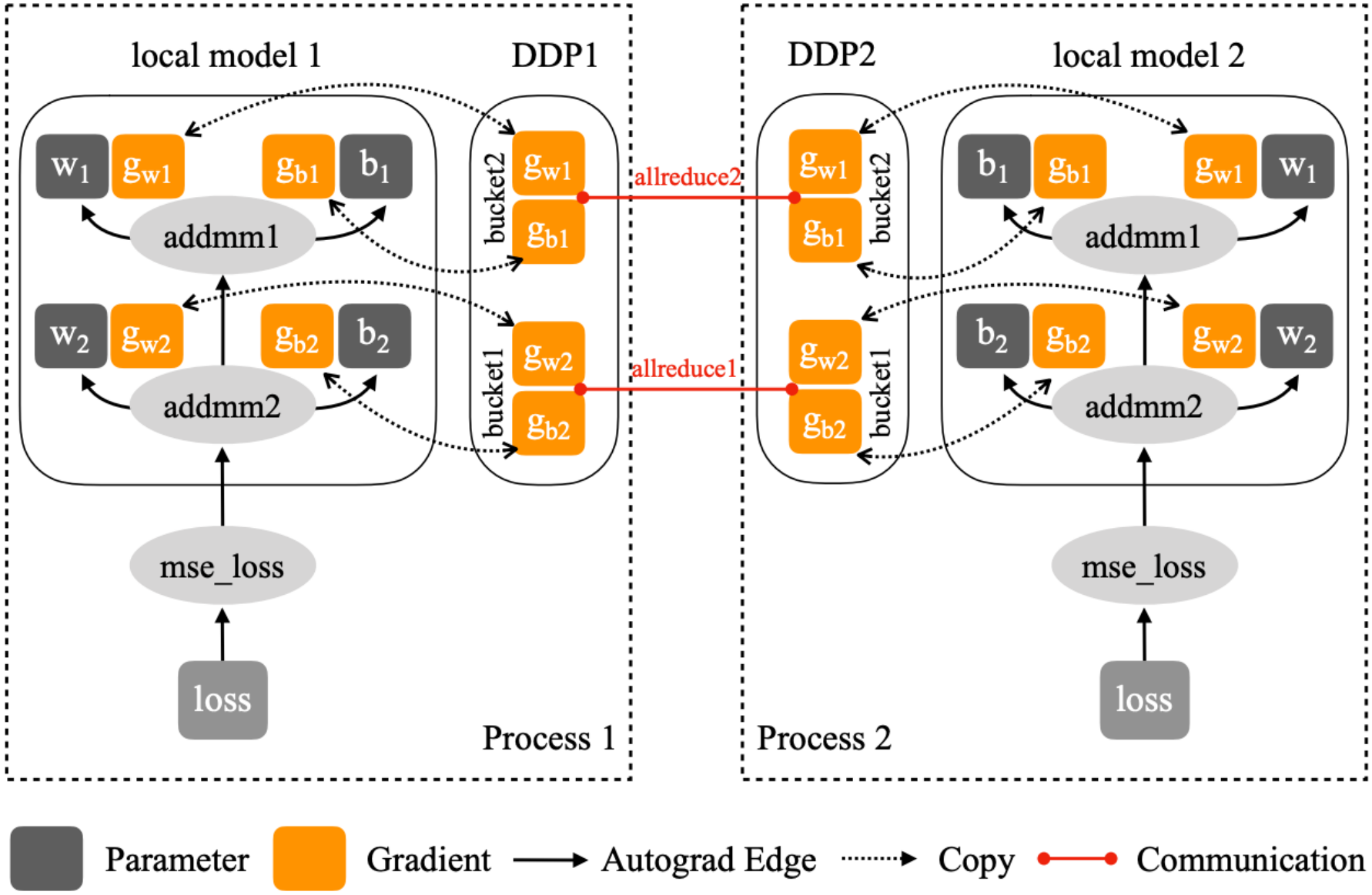


Figure 4: Distributed Gradient Reduction

Gradient accumulation

```
ddp = DistributedDataParallel(net)
with ddp.no_sync():
  for inp, exp in zip(inputs, expected_outputs):
    # no synchronization, accumulate grads
    loss_fn(ddp(inp), exp).backward()
# synchronize grads
loss_fn(ddp(another_inp), another_exp).backward()
opt.step()
```

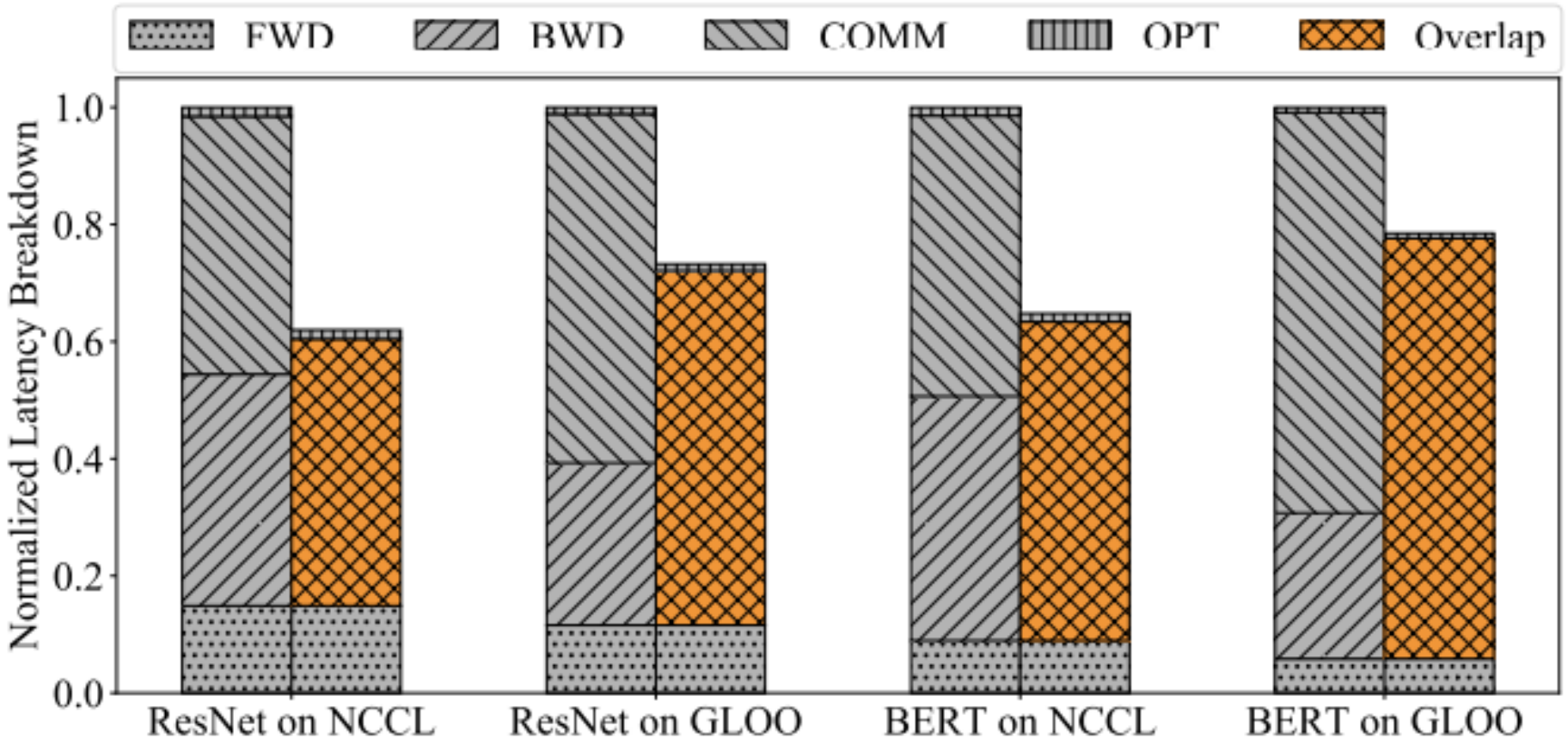
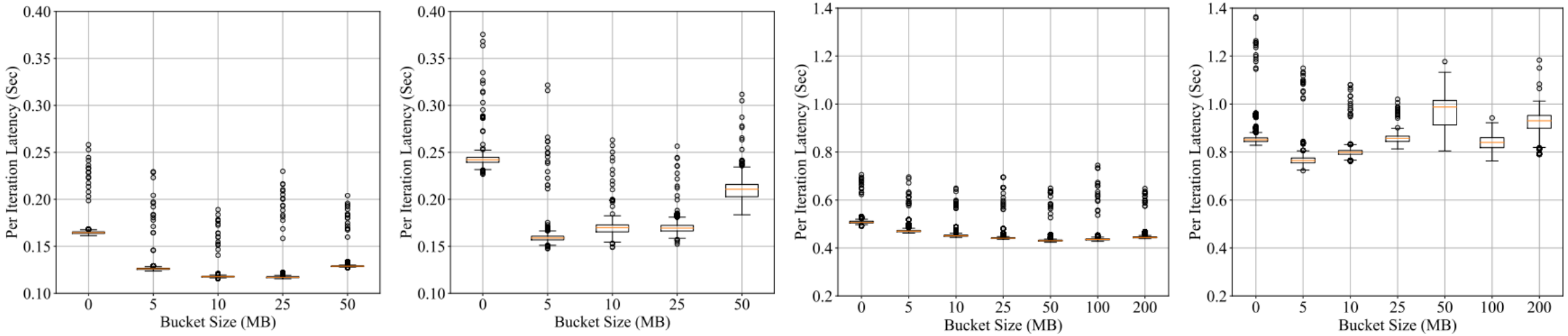
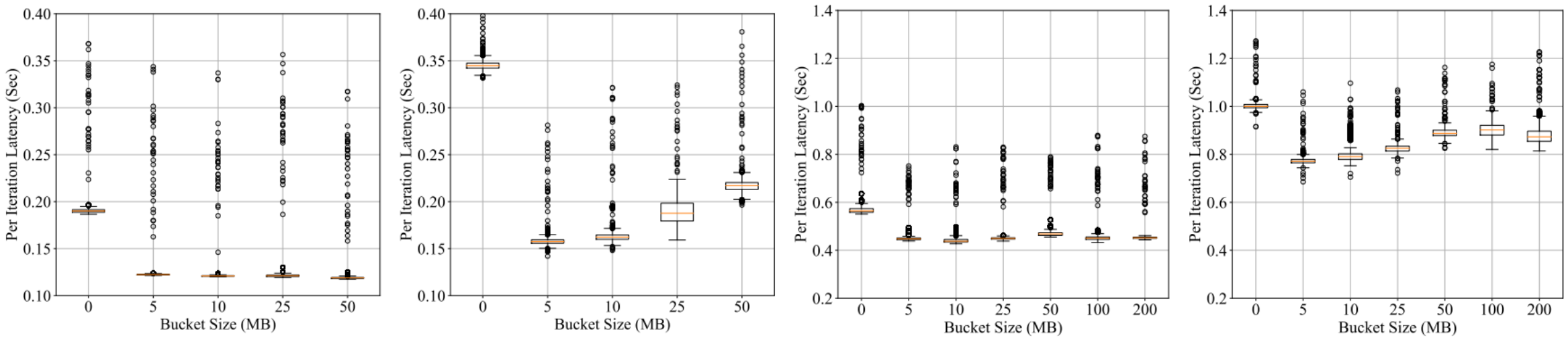


Figure 6: Per Iteration Latency Breakdown



(a) ResNet50 on NCCL (b) ResNet50 on Gloo (c) BERT on NCCL (d) BERT on Gloo

Figure 7: Per Iteration Latency vs Bucket Size on 16 GPUs



(a) ResNet50 on NCCL (b) ResNet50 on Gloo (c) BERT on NCCL (d) BERT on Gloo

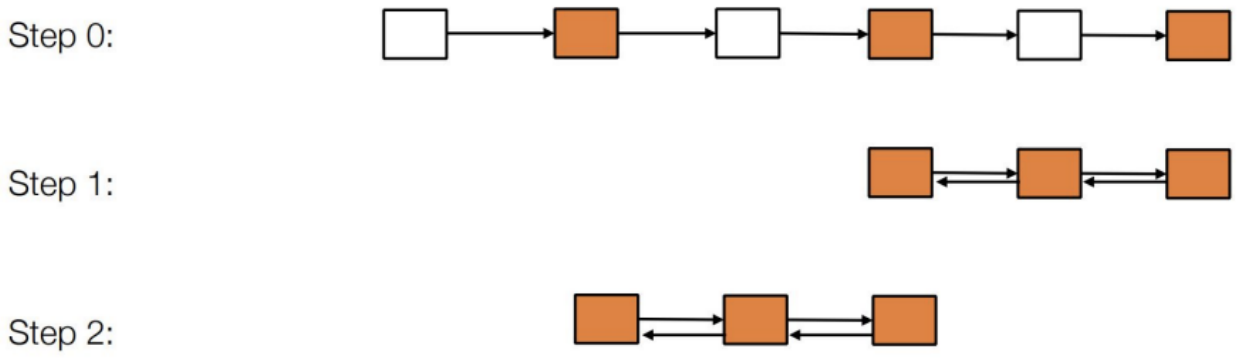
Figure 8: Per Iteration Latency vs Bucket Size on 32 GPUs

Technique #0: Low-precision numbers

Technical Specifications		
	H100 SXM	**H100 NVL**
FP64	34 teraFLOPS	30 teraFLOPS
FP64 Tensor Core	67 teraFLOPS	60 teraFLOPS
FP32	67 teraFLOPS	60 teraFLOPS
TF32 Tensor Core*	989 teraFLOPS	835 teraFLOPS
BFLOAT16 Tensor Core*	1,979 teraFLOPS	1,671 teraFLOPS
FP16 Tensor Core*	1,979 teraFLOPS	1,671 teraFLOPS
FP8 Tensor Core*	3,958 teraFLOPS	3,341 teraFLOPS
INT8 Tensor Core*	3,958 TOPS	3,341 TOPS
GPU Memory	80GB	94GB
GPU Memory Bandwidth	3.35TB/s	3.9TB/s
Decoders	7 NVDEC 7 JPEG	7 NVDEC 7 JPEG
Max Thermal Design Power (TDP)	Up to 700W (configurable)	350-400W (configurable)
Multi-Instance GPUs	Up to 7 MIGs @ 10GB each	Up to 7 MIGS @ 12GB each
Form Factor	SXM	PCIe dual-slot air-cooled
Interconnect	**NVIDIA NVLink™:** 900GB/s **PCIe Gen5:** 128GB/s	**NVIDIA NVLink:** 600GB/s **PCIe Gen5:** 128GB/s

Technique #1: Tensor rematerialization

Reduce activation memory

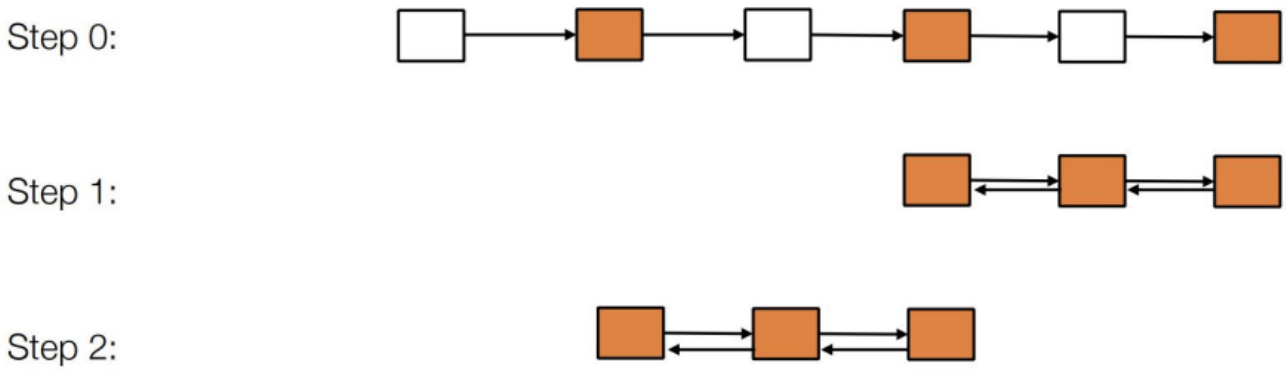


Observation

- The activation is not needed again until the backward pass comes
- Discard some of them and recompute the missing intermediate nodes in small segments

Technique #1: Tensor rematerialization

Reduce activation memory



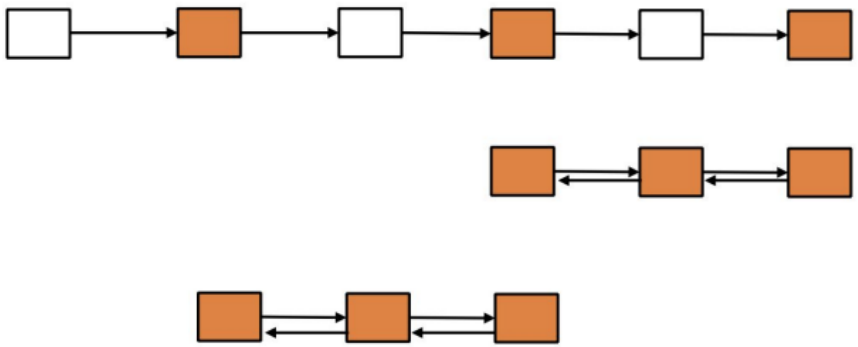
- Extreme case: discord nothing
  - Memory ++, compute --
- Extreme case: discard all and recompute for each layer
  - Memory --, compute++

Technique #1: Tensor rematerialization

Reduce activation memory

Forward computation

Gradient per segment
with re-computation



For a N layer neural network,
if we checkpoint every K layers

$$Memory\ cost = O\left(\frac{N}{K}\right) + O(K)$$

Checkpoint cost

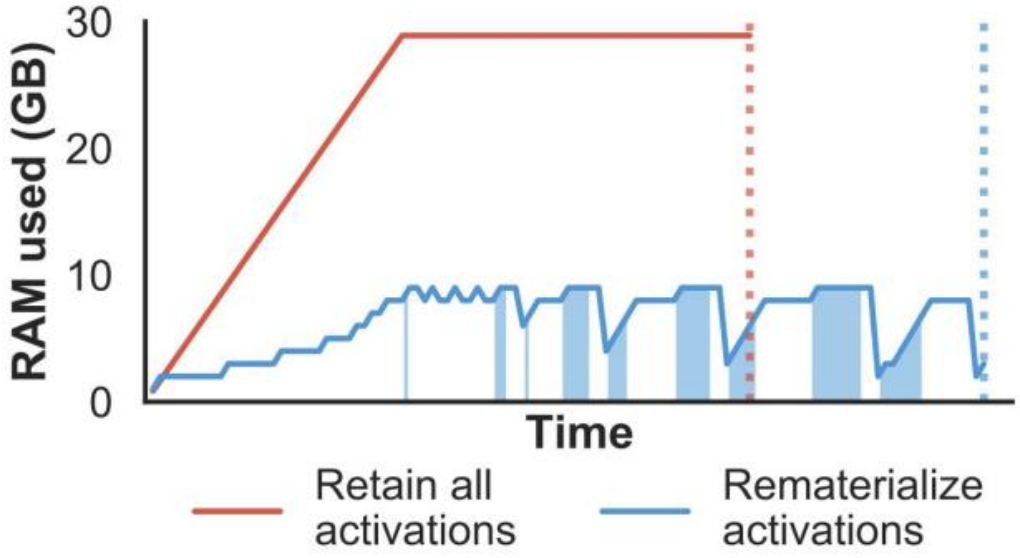
Re-computation cost

Pick $K = \sqrt{N}$

Q: what is the total recomputation cost?

Technique #1: Tensor rematerialization

Memory dynamics

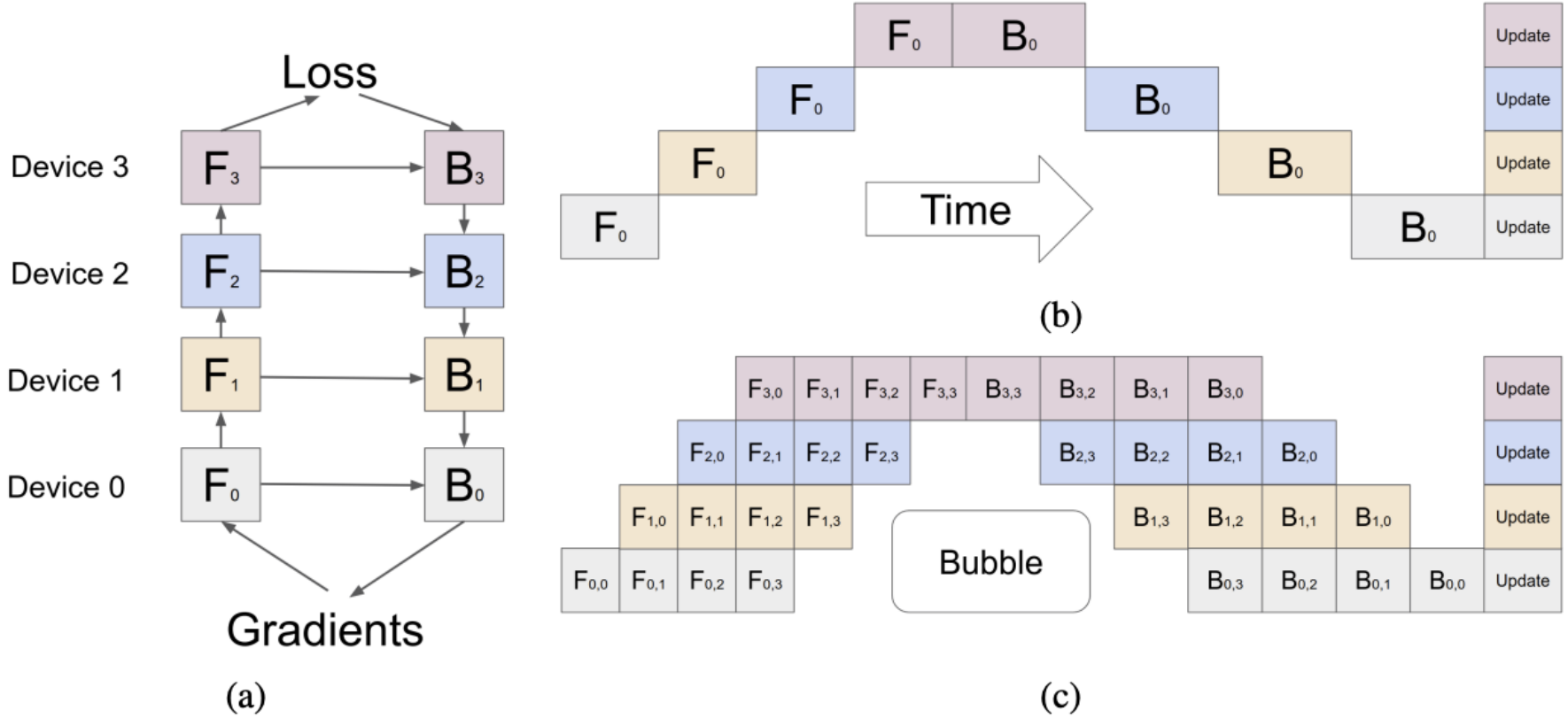


Technique #2: Model parallelism

- A large model cannot fit into a single GPU's memory
  - Need to split a model into multiple GPUs' memory
- Pipeline parallelism
- Tensor model parallelism

GPipe

Figure 2: (a) An example neural network with sequential layers is partitioned across four accelerators. F_k is the composite forward computation function of the k-th cell. B_k is the back-propagation function, which depends on both B_{k+1} from the upper layer and F_k. (b) The naive model parallelism strategy leads to severe under-utilization due to the sequential dependency of the network. (c) Pipeline parallelism divides the input mini-batch into smaller micro-batches, enabling different accelerators to work on different micro-batches simultaneously. Gradients are applied synchronously at the end.



Megatron

$$Y = \text{GeLU}(XA) \tag{1}$$

One option to parallelize the GEMM is to split the weight matrix A along its rows and input X along its columns as:

$$X = [X_1, X_2],\ A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}. \tag{2}$$

This partitioning will result in $Y = \text{GeLU}(X_1A_1 + X_2A_2)$. Since GeLU is a nonlinear function, $\text{GeLU}(X_1A_1 + X_2A_2) \neq \text{GeLU}(X_1A_1) + \text{GeLU}(X_2A_2)$ and this approach will require a synchronization point before the GeLU function.

Another option is to split A along its columns $A = [A_1, A_2]$. This partitioning allows the GeLU nonlinearity to be independently applied to the output of each partitioned GEMM:

$$[Y_1, Y_2] = [\text{GeLU}(XA_1), \text{GeLU}(XA_2)] \tag{3}$$

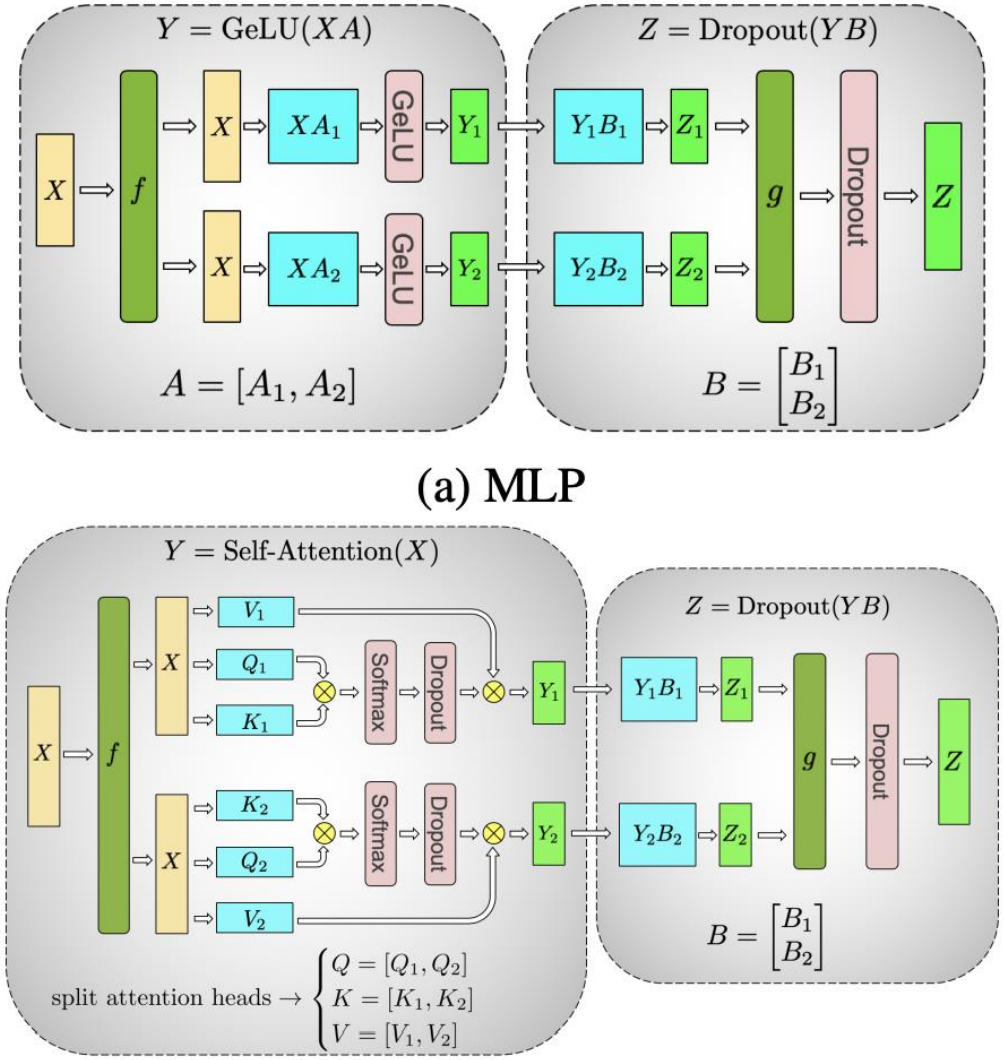


Figure 3. Blocks of Transformer with Model Parallelism. f and g are conjugate. f is an identity operator in the forward pass and all reduce in the backward pass while g is an all reduce in the forward pass and identity in the backward pass.